

The Design and Implementation of a RISC-V processor with an Embedded Encryption Accelerator

Ofek Baribi, Gilad Navok

Instructor: Etay Sela

Overview

- Background.
- Project Definition & System Top-Level & Specifications
- RISC-V Core Microarchitecture.
- The AES Accelerator.
- Integration & Software Interface.

Background

Background: RISC-V & The Acceleration Era

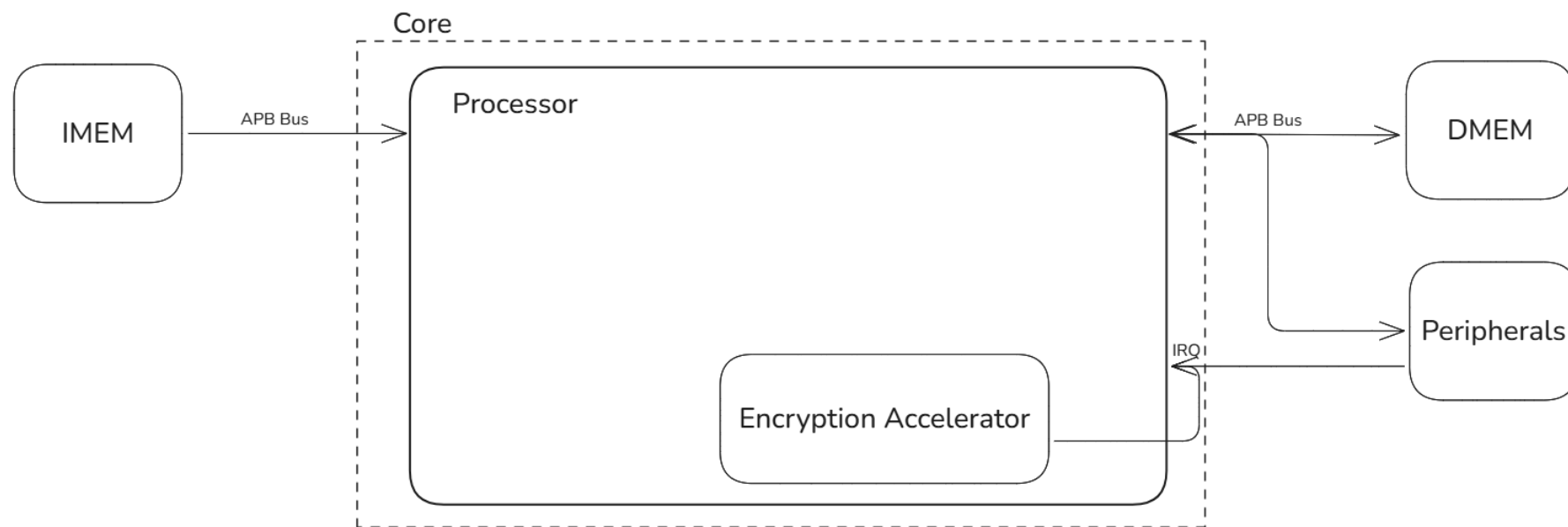
- The RISC-V Ecosystem
 - **Open & Modular:** A royalty-free ISA explicitly designed to support custom, user-defined extensions
 - **The Edge Standard:** Rapidly becoming the dominant architecture for low-power IoT and micro-controllers.
- Hardware Acceleration
 - **Post-Moore's Law:** As transistor scaling slows, performance gains require dedicated, workload-specific hardware.
 - **The Trade-off:** Accelerators provide massive speed-ups, but silicon is expensive—performance must justify the area overhead.

Project Definition

“The Design and Implementation of a RISC-V processor with an Embedded Encryption Accelerator”

- For low power edge devices.
- Three milestones
 - Design and implement a processor.
 - Design and implement an encryption accelerator.
 - Embed the encryption accelerator into the processor.

System Top-Level



Core Architectural Specifications

- Base ISA: Unprivileged RV32I.
- Extensions:
 - Zicsr (Control and Status Registers)
 - Custom Cryptographic Extensions.
- Privilege Modes: Machine-Mode only.
- Full support for asynchronous external and internal interrupts.

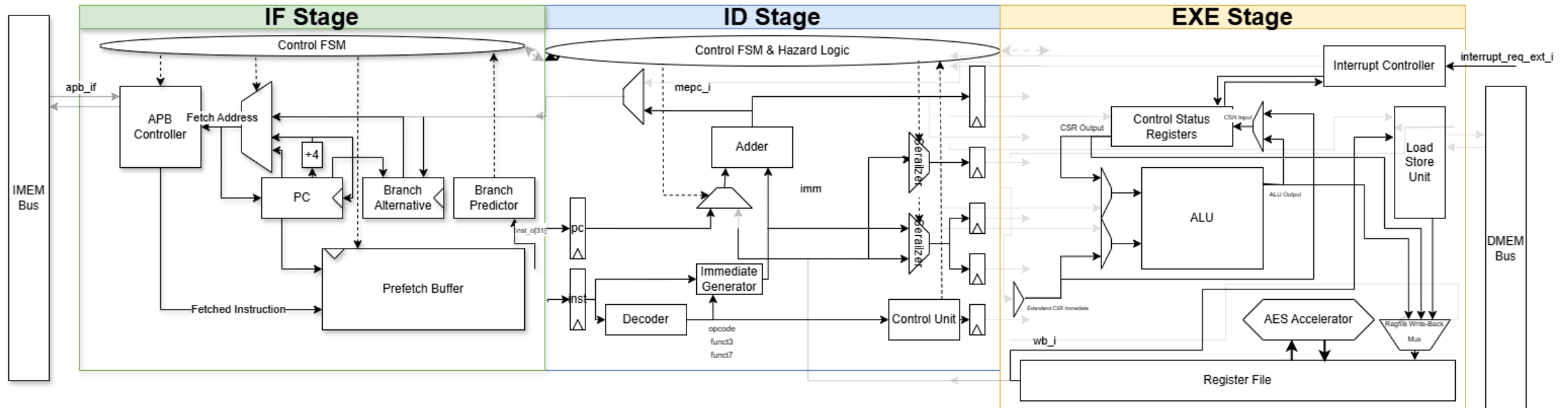
Architectural Constraint: The AMBA APB Interface

- A low-cost interface optimized for minimal power consumption and reduced interface complexity.
- **Every memory transfer takes at least two cycles to complete.**
- Used for communication with both Instruction and Data Memory.
 - Not realistic.
 - But a core design optimized around it, may be.
 - Allows for a lower area, lower-powered design and resource sharing:
 - Two cores can share the same one-cycle fetch bus.
 - One core can use the same memory component and one-cycle fetch bus for both IMEM and DMEM.

RISC-V Core Microarchitecture

Microarchitectural Overview

- In-order three stage pipeline.
 - Instruction Fetch
 - Instruction Decode
 - Execution
- 16-bit data-paths:
 - 16-bit ALU.
 - Register File with one 32 bit read port and one 16 bit write port.



Processor Microarchitecture Design – Instruction Execution

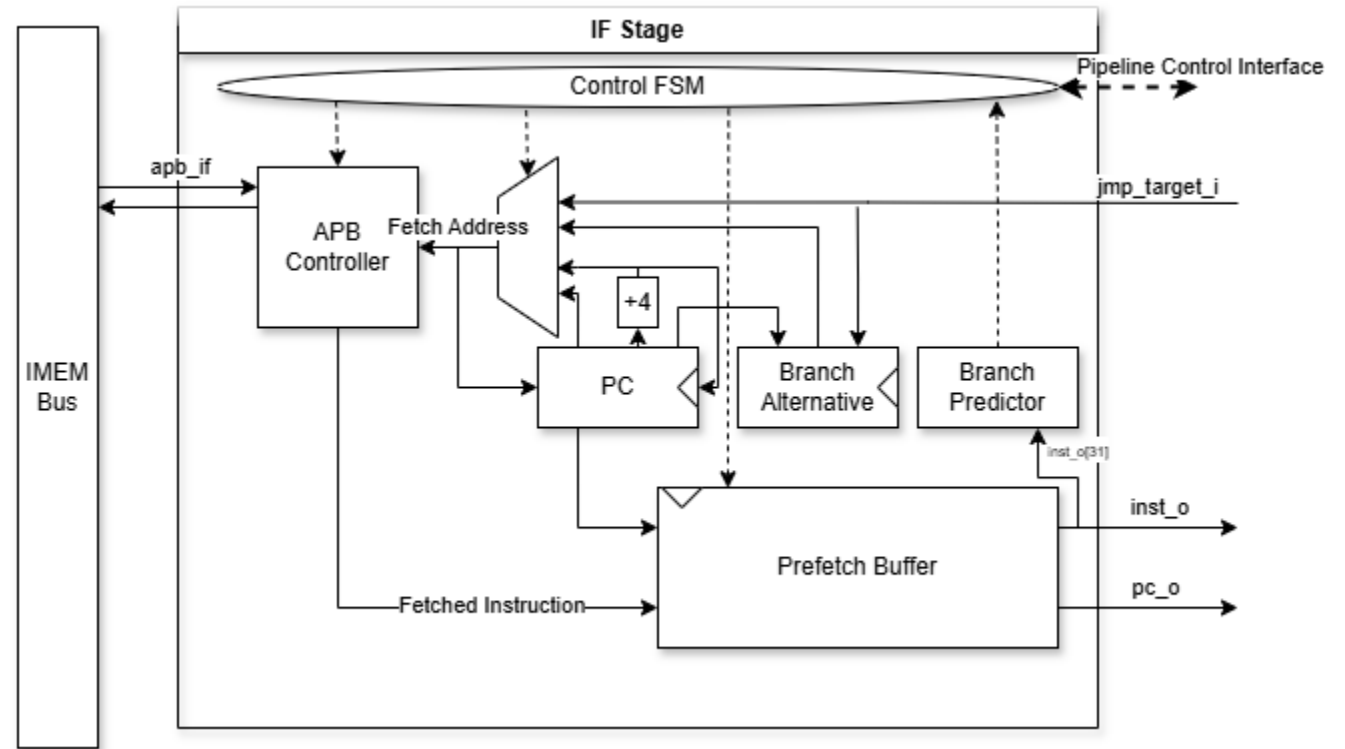
- Staggered pipeline.
- Instructions are executed in 5 cycles:
 - Two fetch cycles.
 - One cycle in ID stage.
 - One cycle in both ID and EXE stages.
 - One cycle in EXE stage.

Cycle/ Instruction	1	2	3	4	5	6	7
I1	IF	IF	ID	ID/ EXE	EXE		
I2			IF	IF	ID	ID/ EXE	EXE
I3					IF	IF	ID

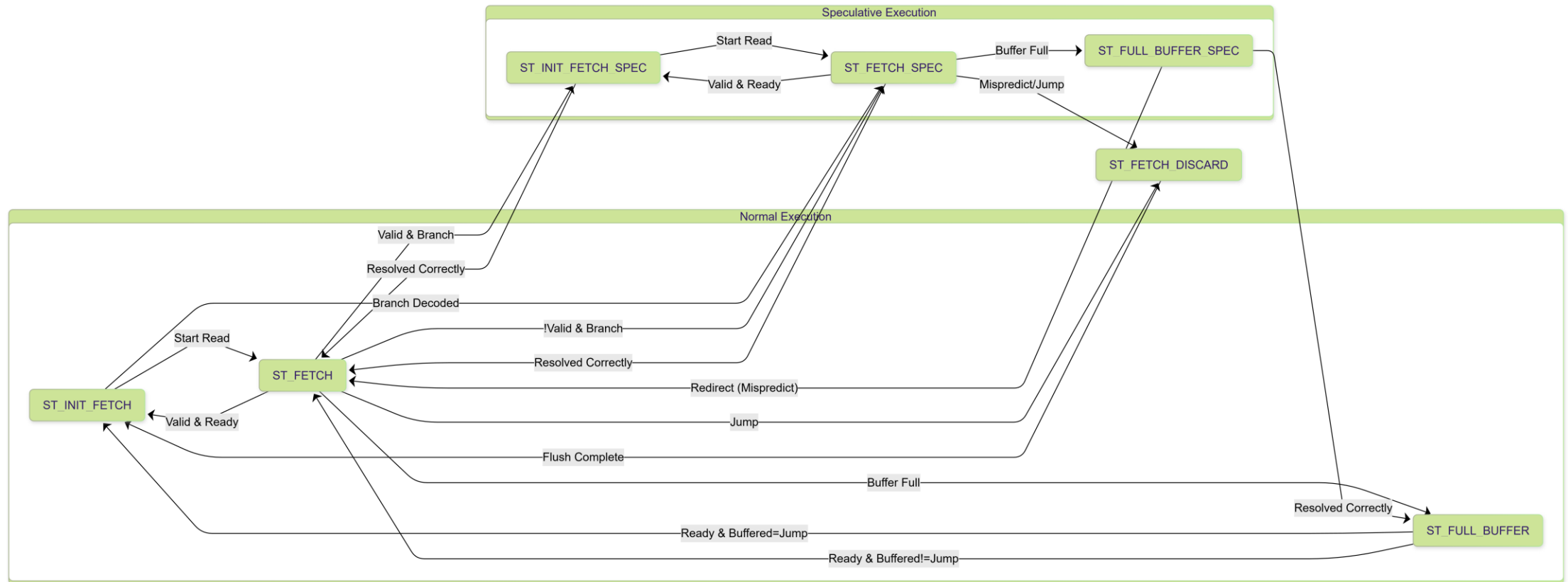
Steady-state
– All stages
populated

Instruction Fetch (IF)

- APB Controller
- PC; Branch Speculation
- Prefetch Buffer.
 - Parametric Size.

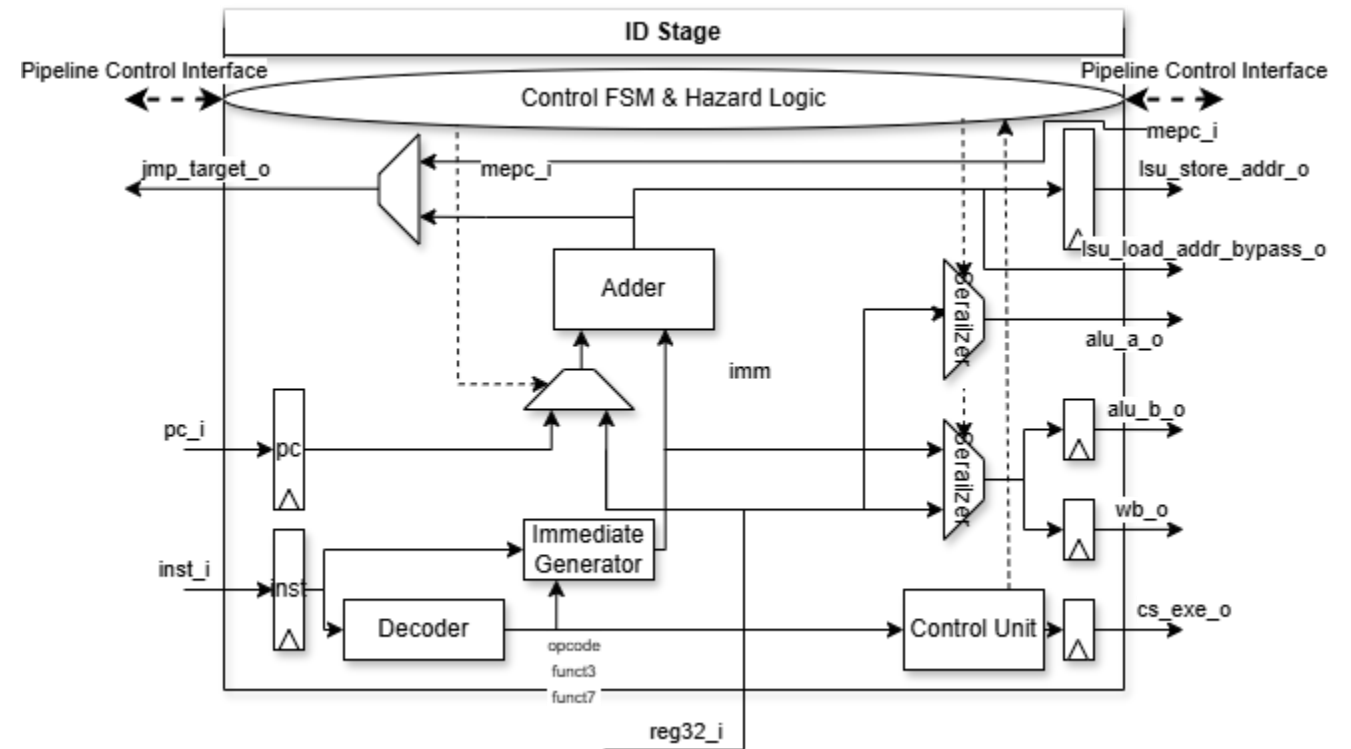


Instruction Fetch Control FSM

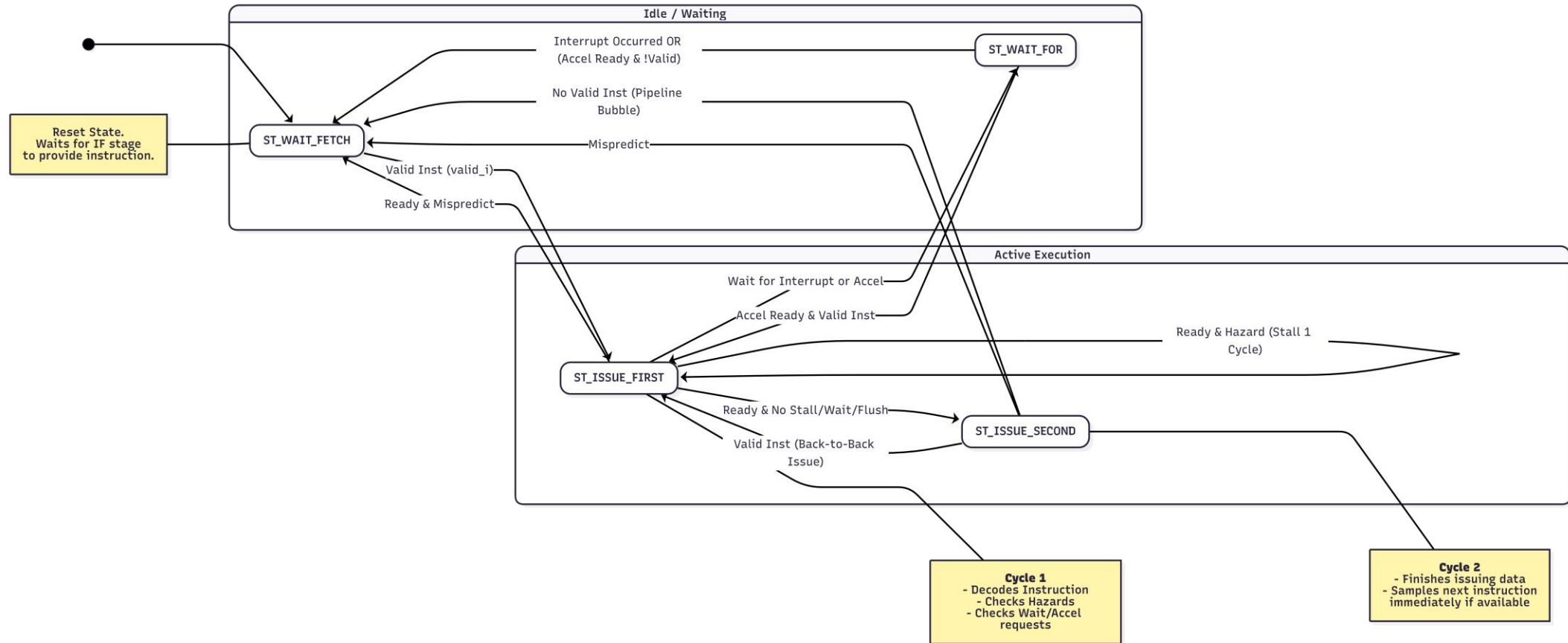


Instruction Decode (ID)

- Decoder.
- 32 bit Adder.
- Control Unit.
- Serializers.

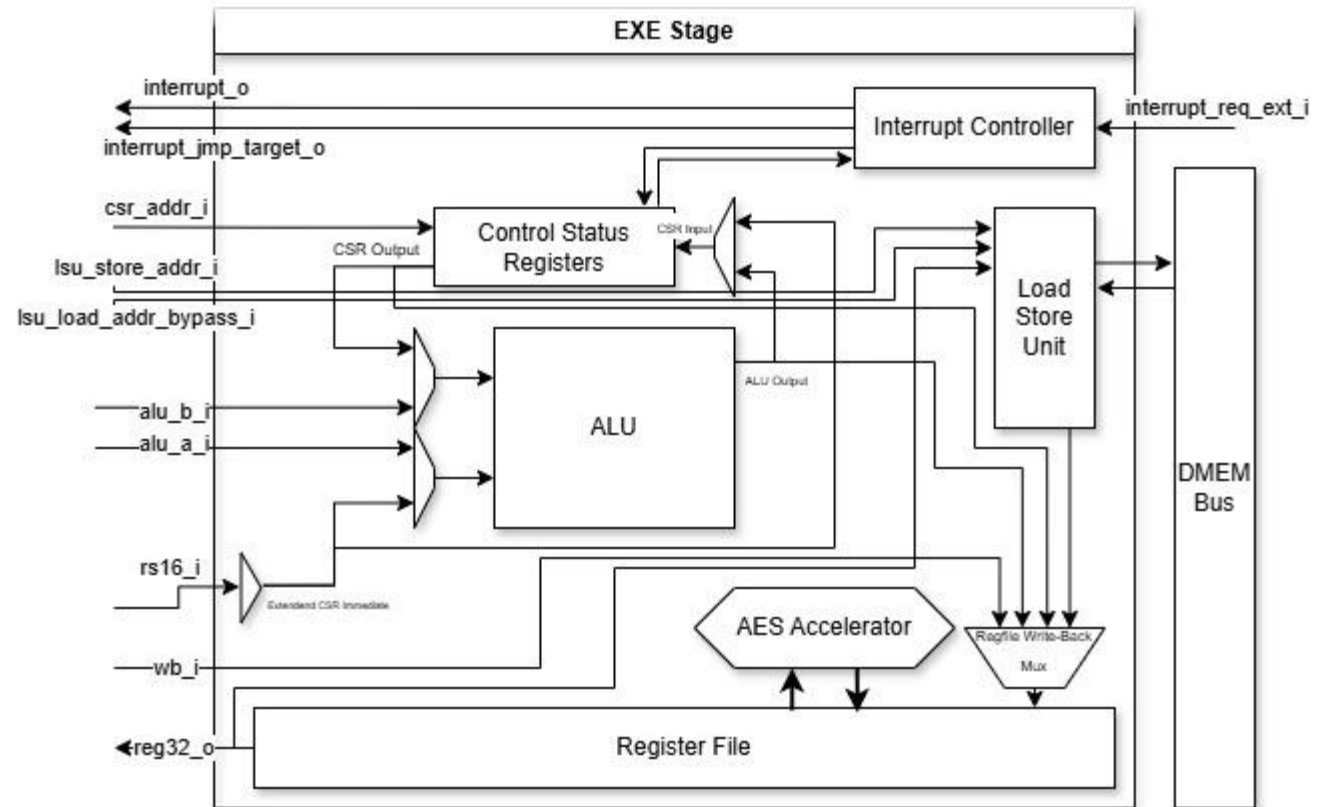


Instruction Decode Control FSM



Execution Stage (EXE)

- ALU; LSU; Register File.
- CSRs.
- Interrupt Controller.
- Accelerator.
- No Control FSM.
 - Slave of Instruction Decode Stage.



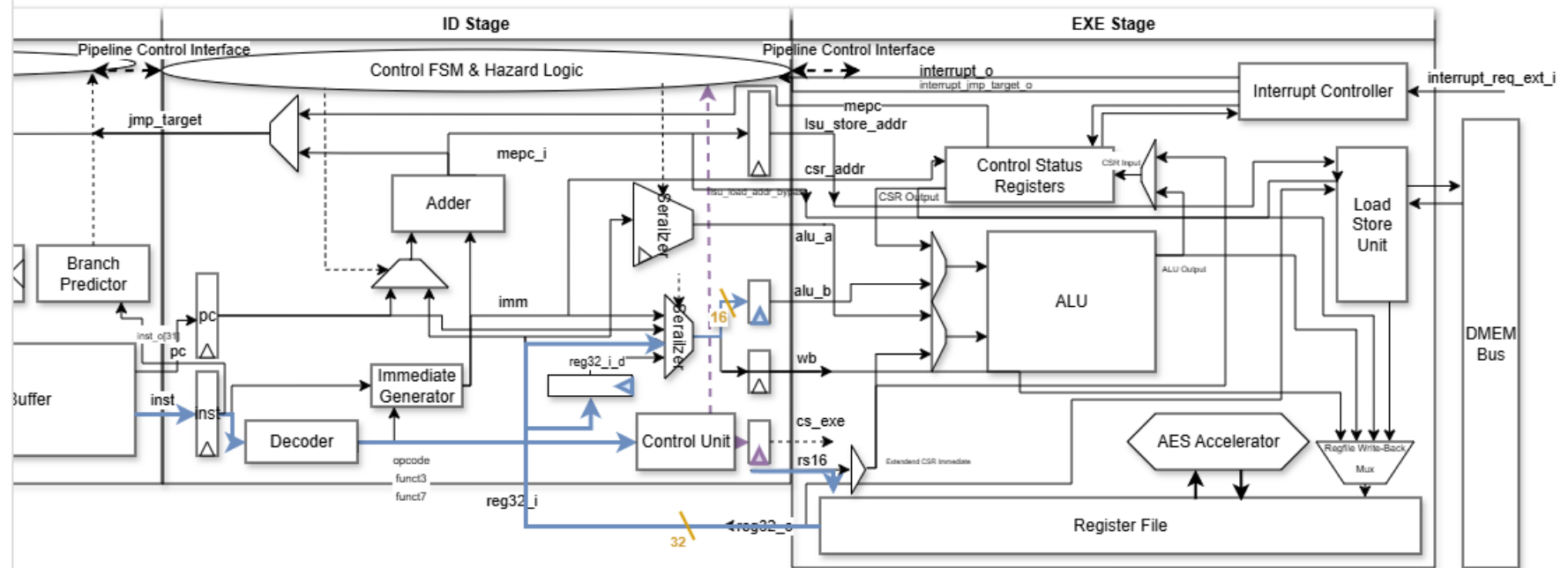
Datapath Walkthrough – ADD - ID

ADD RD, RS1, RS2:

- $RD = RS1 + RS2$

This cycle:

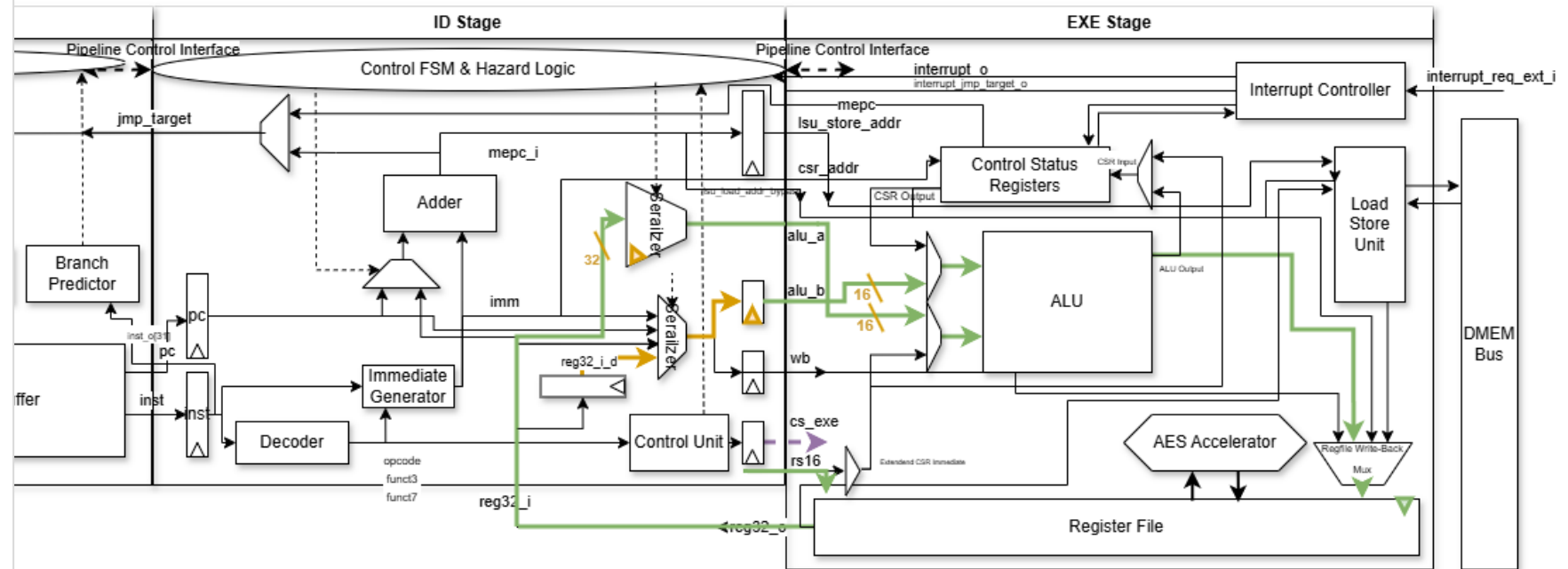
- In ID stage:
 - Generate control signals for both ID and EXE stages.
 - Forward control signals to EXE stage.
- Read RS2
- Forward the first half of RS2 to EXE Stage through alu_b FF.
- Store the second half.



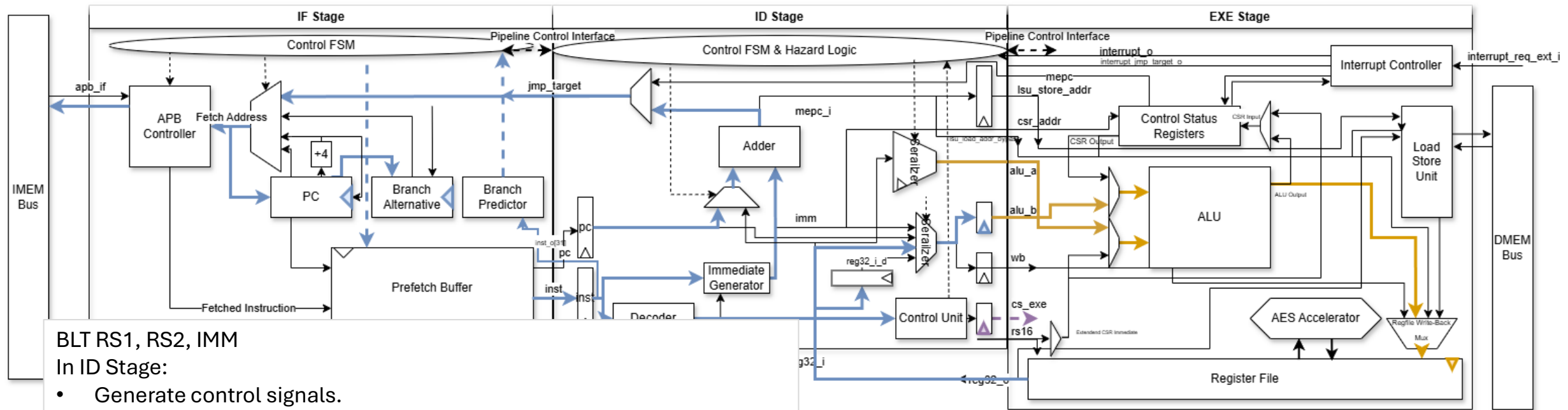
Datapath Walkthrough – ADD – ID/EXE

This cycle:

- In ID stage:
 - Read RS1.
 - Feed its first half to alu_a
 - Store its second half.
 - Forward the second half of RS2 to EXE Stage through alu_b
- In EXE stage:
 - Add first halves.
 - Store first half of the result to the first half of RD.
 - Store carry bit.



Datapath Walkthrough – BLT – ID



BLT RS1, RS2, IMM

In ID Stage:

- Generate control signals.
- Compute the jump target address (PC + IMM).
- Start the same procedure of reading RS1 and RS2 and forwarding to EXE stage for comparison

In IF Stage:

- Branch predicted taken – Reset Prefetch buffer and start fetching jump target instruction.

ADD RD, RS1, RS2

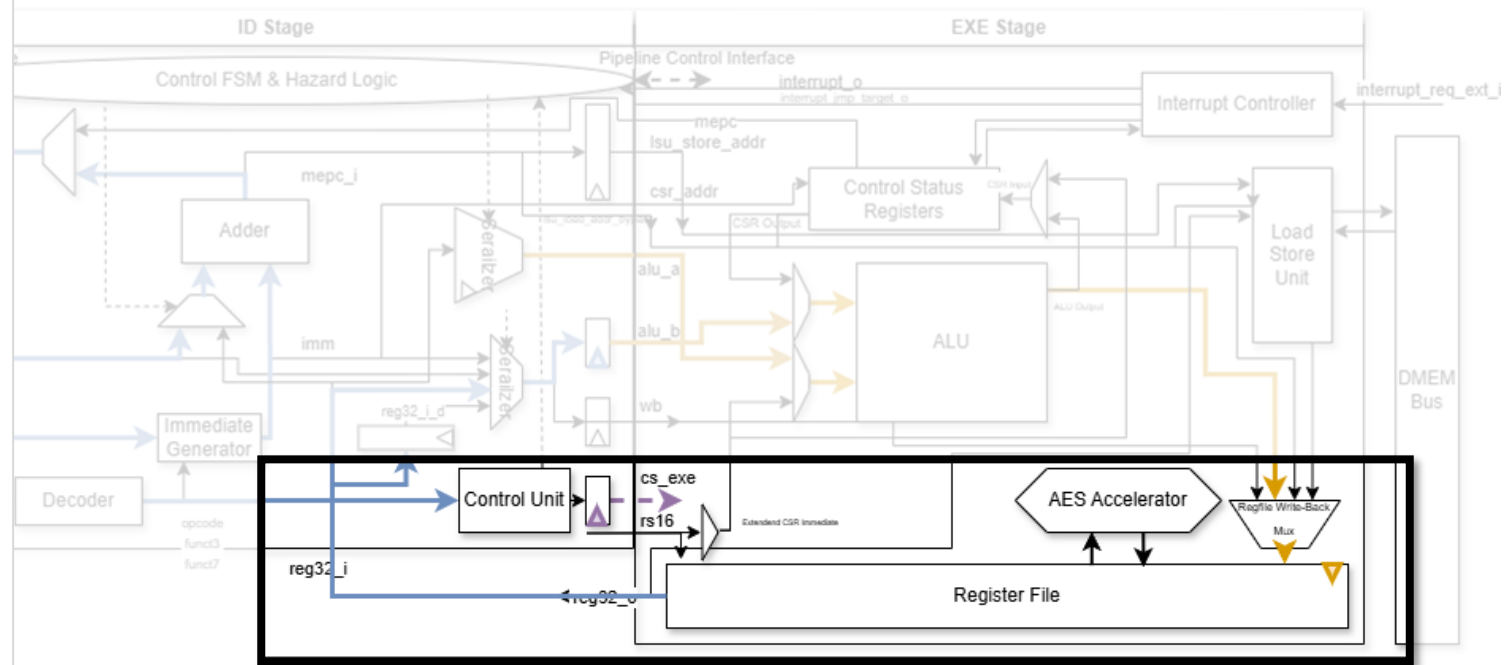
In EXE Stage:

- Add second halves
- Store second half of the result to the second half of RD.

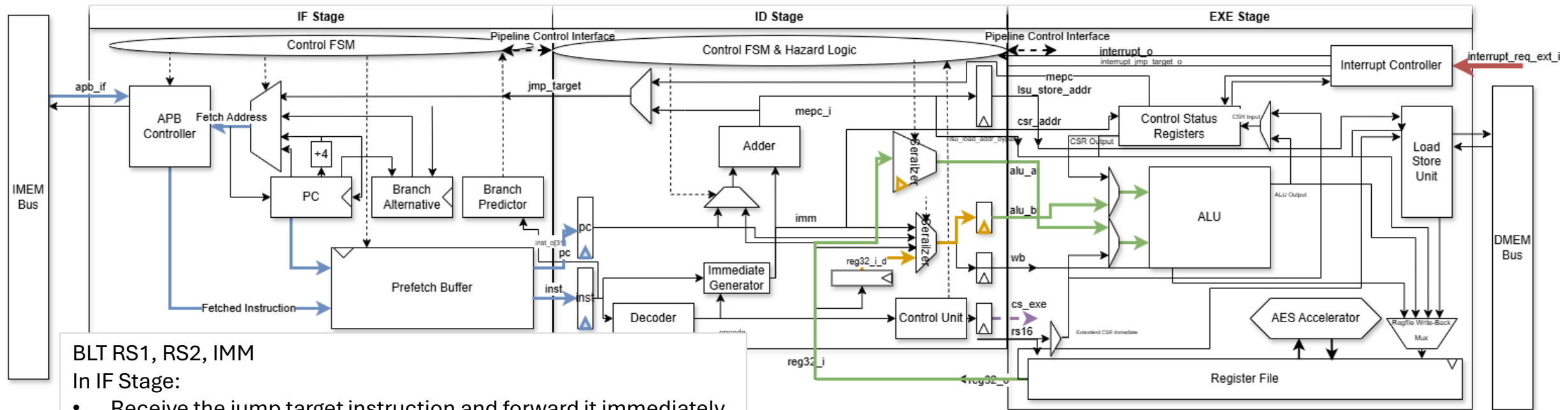
Datapath Walkthrough – RAW Hazards

Possible read after write hazard.

- Common and safe –
 - Read from lower half,
 - Write to upper half.
- Not safe:
 - Full reads.
 - Instructions that read from upper half first
 - F.e BLT for possible early comparison verdict.
- Resolved using stalls mostly.



Datapath Walkthrough – BLT – ID/EXE



BLT RS1, RS2, IMM

In IF Stage:

- Receive the jump target instruction and forward it immediately to ID stage.

In ID Stage:

- Continue half forwarding procedure.

BLT RS1, RS2, IMM

In EXE Stage:

- Compare first halves. Signal IF stage incase of early resolution.

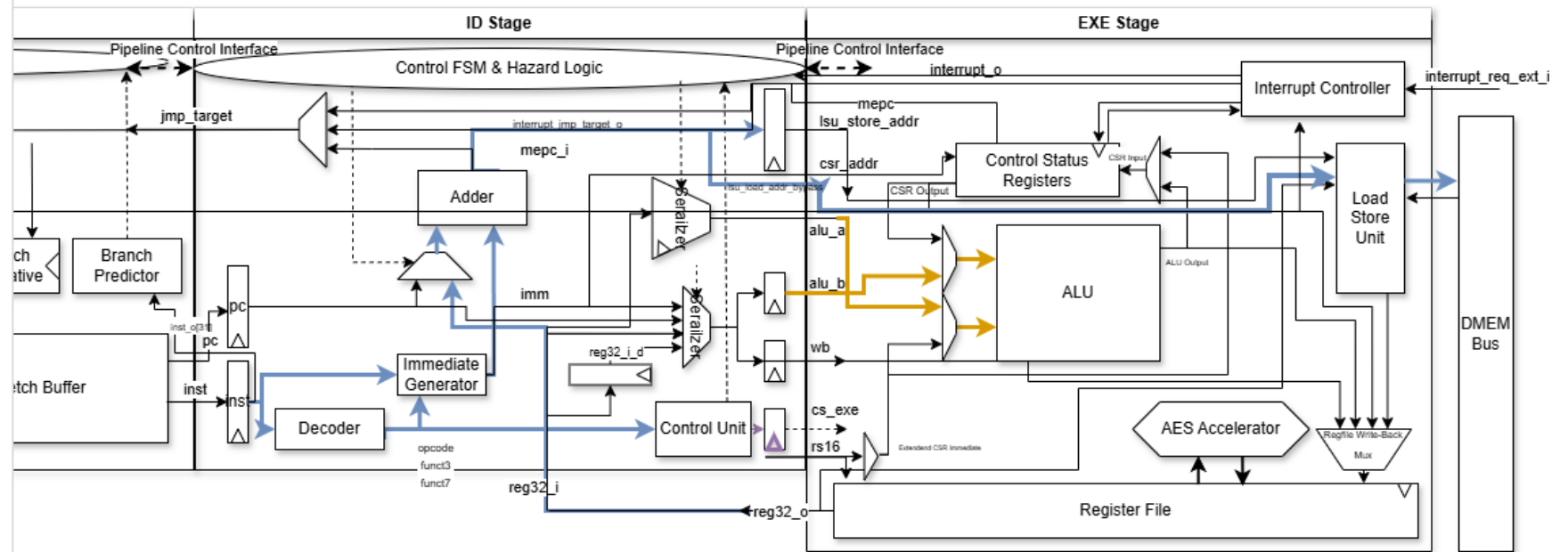
Datapath Walkthrough – Load - ID

LW RD, RS1, IMM:

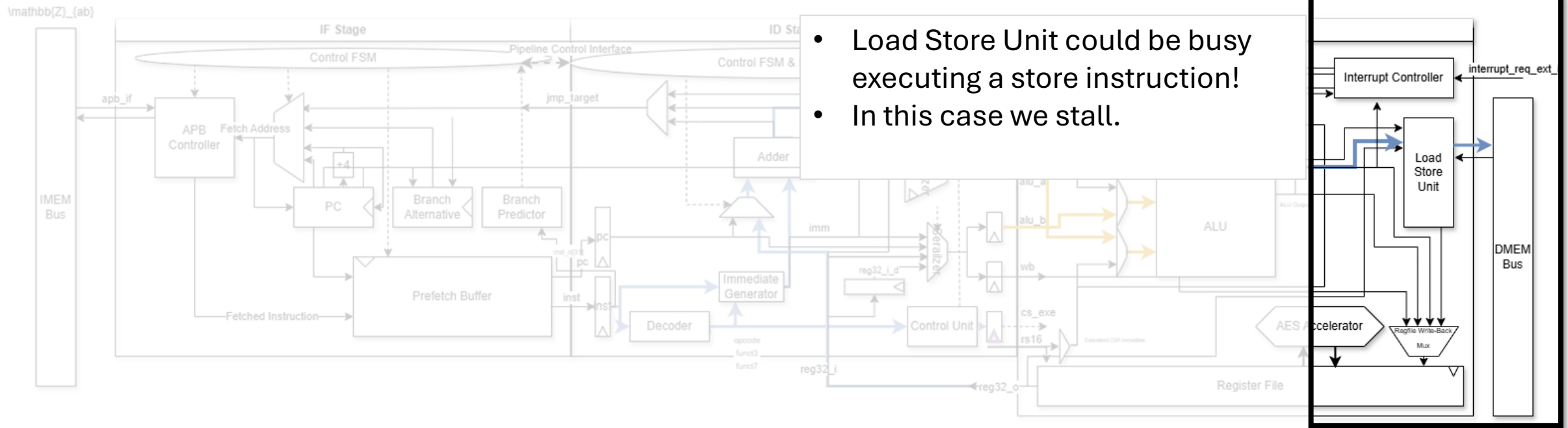
- $RD = DMEM[RS1 + IMM]$

This cycle:

- In ID stage:
 - Generate control signals for both ID and EXE stages.
 - Read RS1.
 - Generate IMM.
 - Calculate load address: $RS1 + IMM$
 - Start APB transfer this cycle!



Datapath Walkthrough – Load-Store Hazard



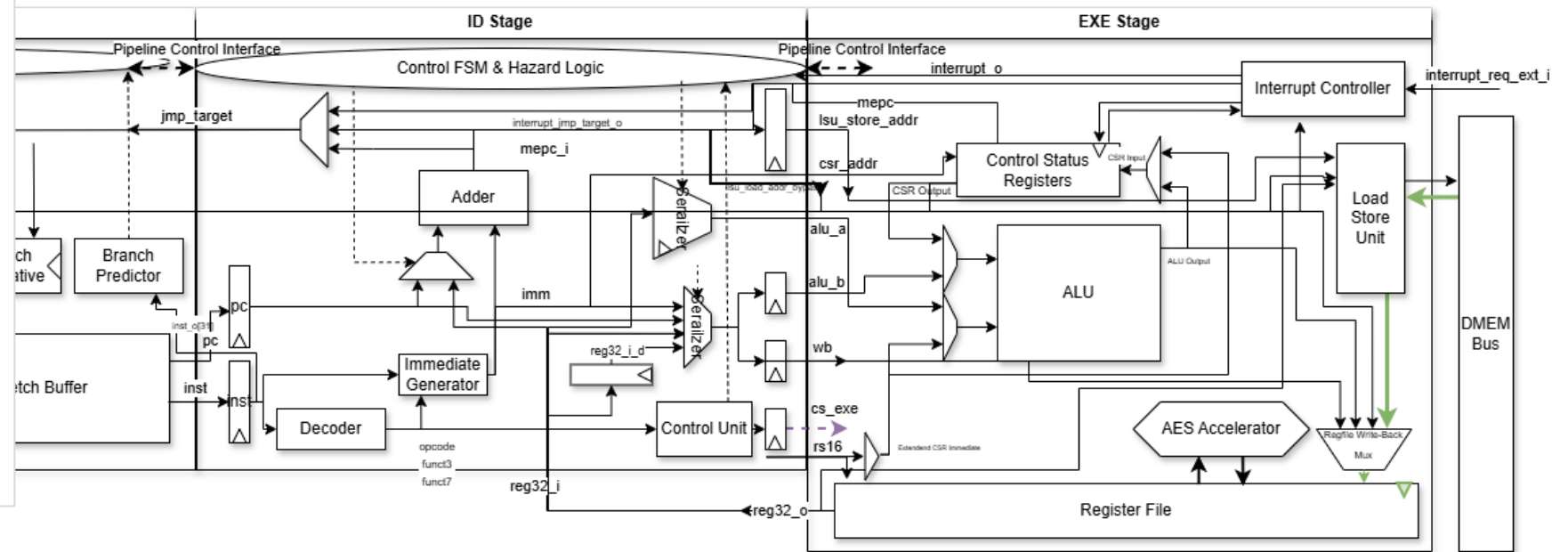
Datapath Walkthrough – Load – ID/EXE

LW RD, RS1, IMM:

- $RD = DMEM[RS1 + IMM]$

This cycle:

- In EXE stage
 - Receive word from DMEM.
 - Write one half to the Register File and store the other half.



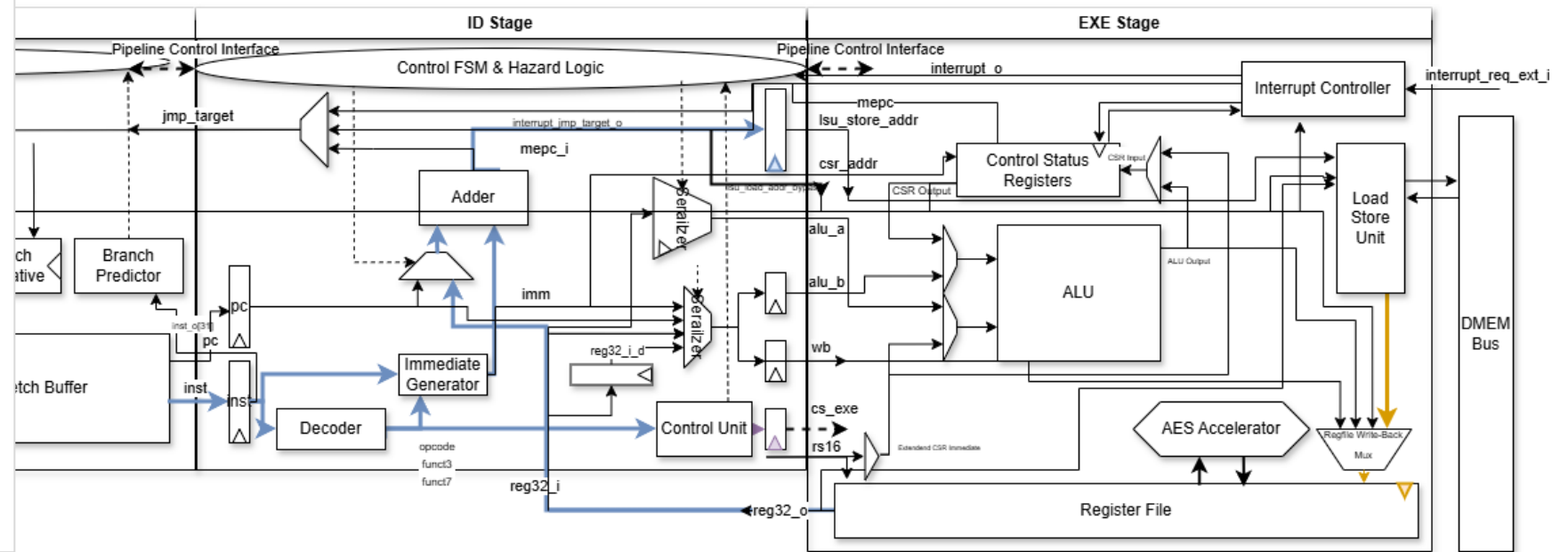
Datapath Walkthrough – Store - ID

SW RS1, RS2, IMM:

- $\text{DMEM}[\text{RS1} + \text{IMM}] = \text{RS2}$

This cycle:

- In ID stage:
 - Generate control signals for both ID and EXE stages.
 - Read RS1.
 - Generate IMM.
 - Calculate load address: $\text{RS1} + \text{IMM}$ and forward it.



- In EXE stage
 - Write the second half to the Register File.

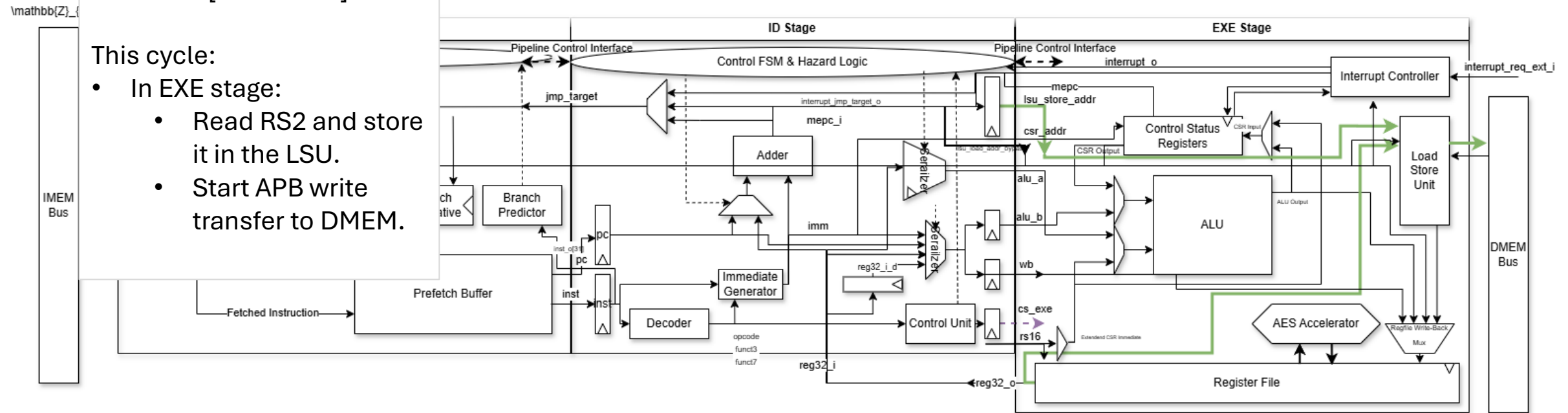
Datapath Walkthrough – Store – ID/EXE

SW RS1, RS2, IMM:

- $\text{DMEM}[\text{RS1} + \text{IMM}] = \text{RS2}$

This cycle:

- In EXE stage:
 - Read RS2 and store it in the LSU.
 - Start APB write transfer to DMEM.



The AES Accelerator

What is AES?

- Symmetric block cipher standardized by NIST (FIPS 197).
- Processes 128-bit data blocks; key sizes: 128 / 192 / 256 bits.
- This project implements AES-128 (128-bit key, 10 rounds).
- Commonly used in secure communication and storage systems.

AES Round Transformations (Encryption vs. Decryption)

Encryption (forward)

- AddRoundKey (XOR with round key)
- SubBytes (S-Box substitution)
- ShiftRows (row rotation)
- MixColumns (GF(2⁸) column mixing)
- Final round skips MixColumns

Decryption (inverse)

- InvShiftRows (reverse rotation)
- InvSubBytes (inverse S-Box)
- AddRoundKey (XOR with round key)
- InvMixColumns (inverse mixing)
- Final inverse round skips InvMixColumns

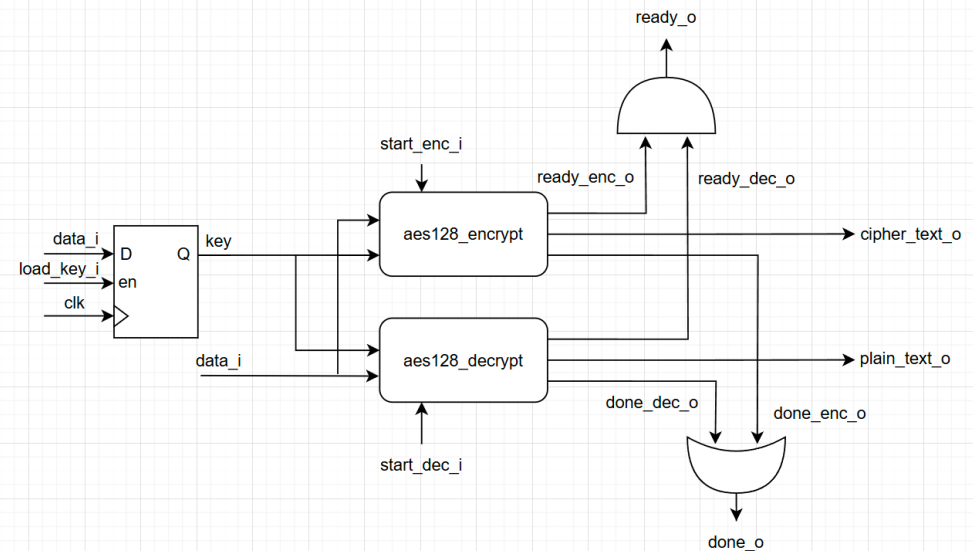


One standard round (AES-128 has 9 standard rounds + final round)

Top-Level Architecture of AES Core

Interfaces

- Key register loaded via load_key / data_i.
- start_enc_i triggers encryption; start_dec_i triggers decryption.
- cipher_text_o is produced by aes128_encrypt.
- plain_text_o is produced by aes128_decrypt.
- ready_o indicates idle/ready; done_o asserts when operation finishes.

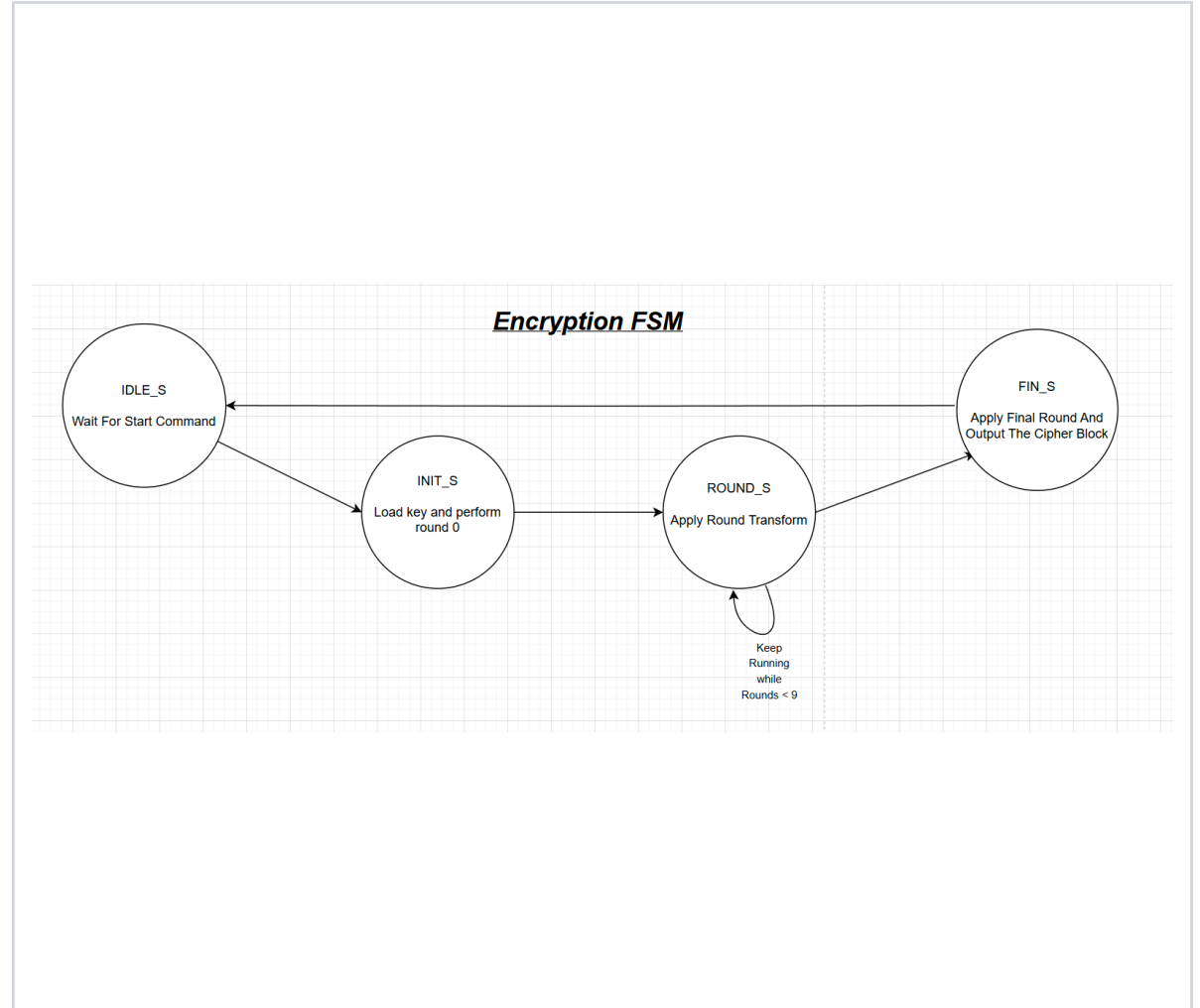


Block-level view (key register + encrypt/decrypt cores)

Control — Encryption FSM

Key states:

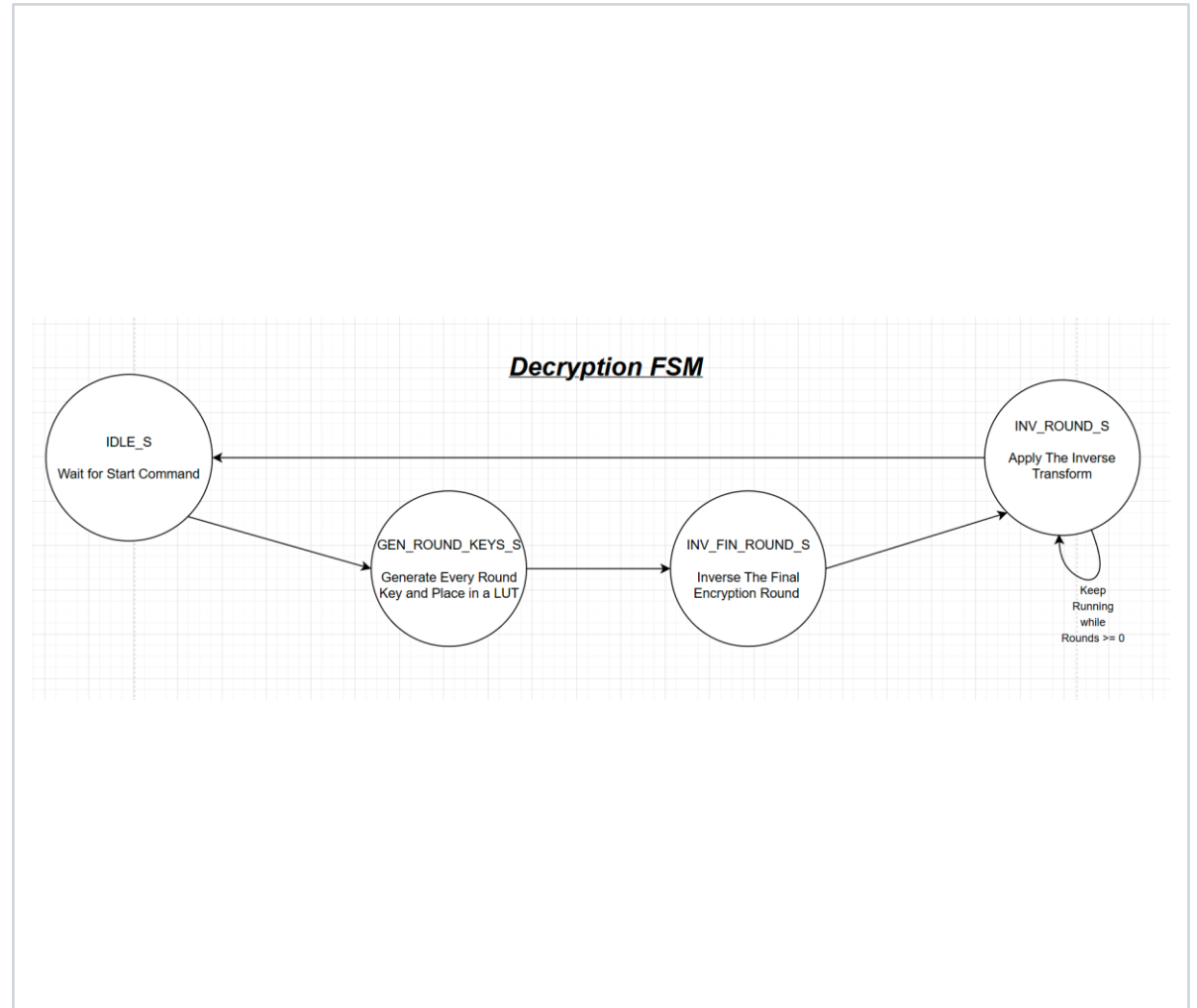
- IDLE_S: wait for start_enc_i
- INIT_S: load key + round 0 (AddRoundKey)
- ROUND_S: run rounds 1...9
- FIN_S: final round, output ciphertext



Control — Decryption FSM

Key states:

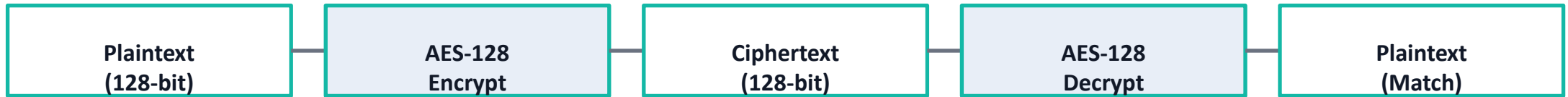
- IDLE_S: wait for start_dec_i
- GEN_ROUND_KEYS_S: generate all round keys
- INV_FIN_ROUND_S: inverse of final round
- INV_ROUND_S: run inverse rounds until done



Verification — Testbench Method

Goal

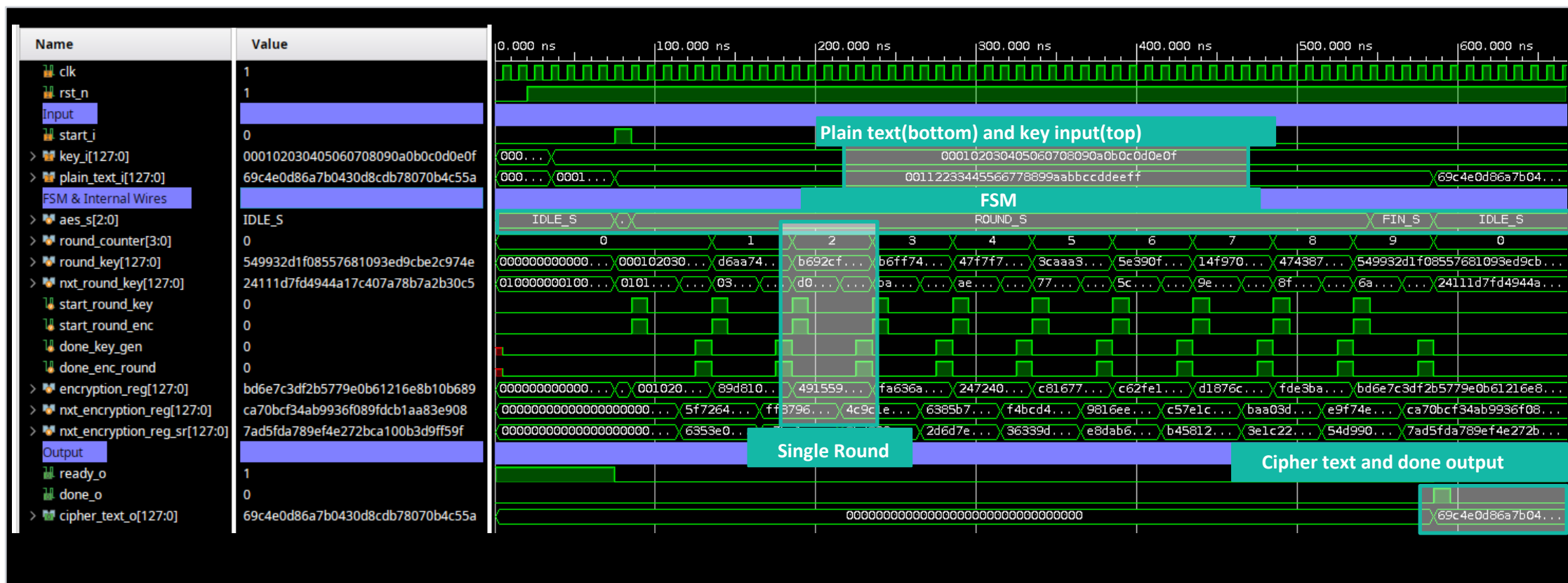
Show round-trip correctness: plaintext → encrypt → ciphertext → decrypt → original plaintext



Steps

- Reset DUT; load 128-bit key.
- Apply plaintext, pulse start_enc_i; wait for done_o.
- Capture cipher_text_o (encryption result).
- Feed ciphertext into decrypt core, pulse start_dec_i; wait done_o.
- Check plain_text_o equals the original plaintext (self-check).

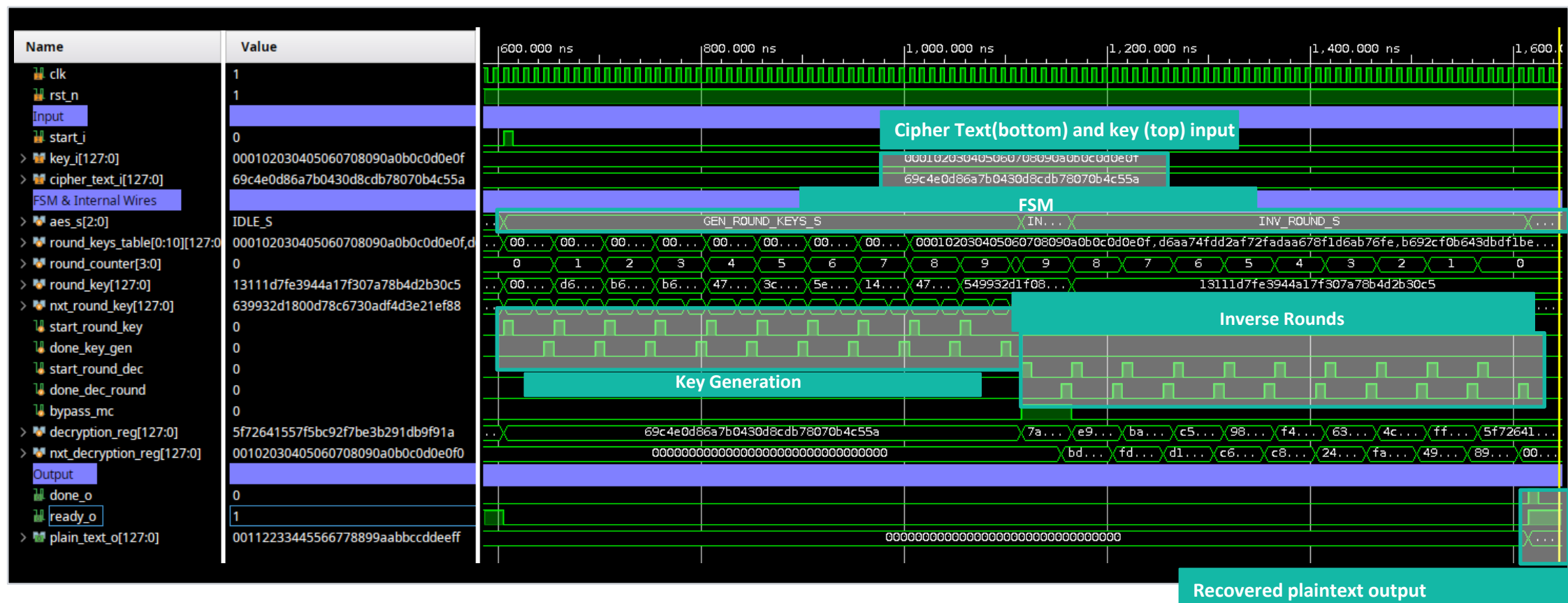
Simulation Results — Encryption



Observed:

- start_enc_i pulses once; core advances through rounds and then asserts done_o.
- cipher_text_o becomes valid at the end of FIN_S.

Simulation Results — Decryption



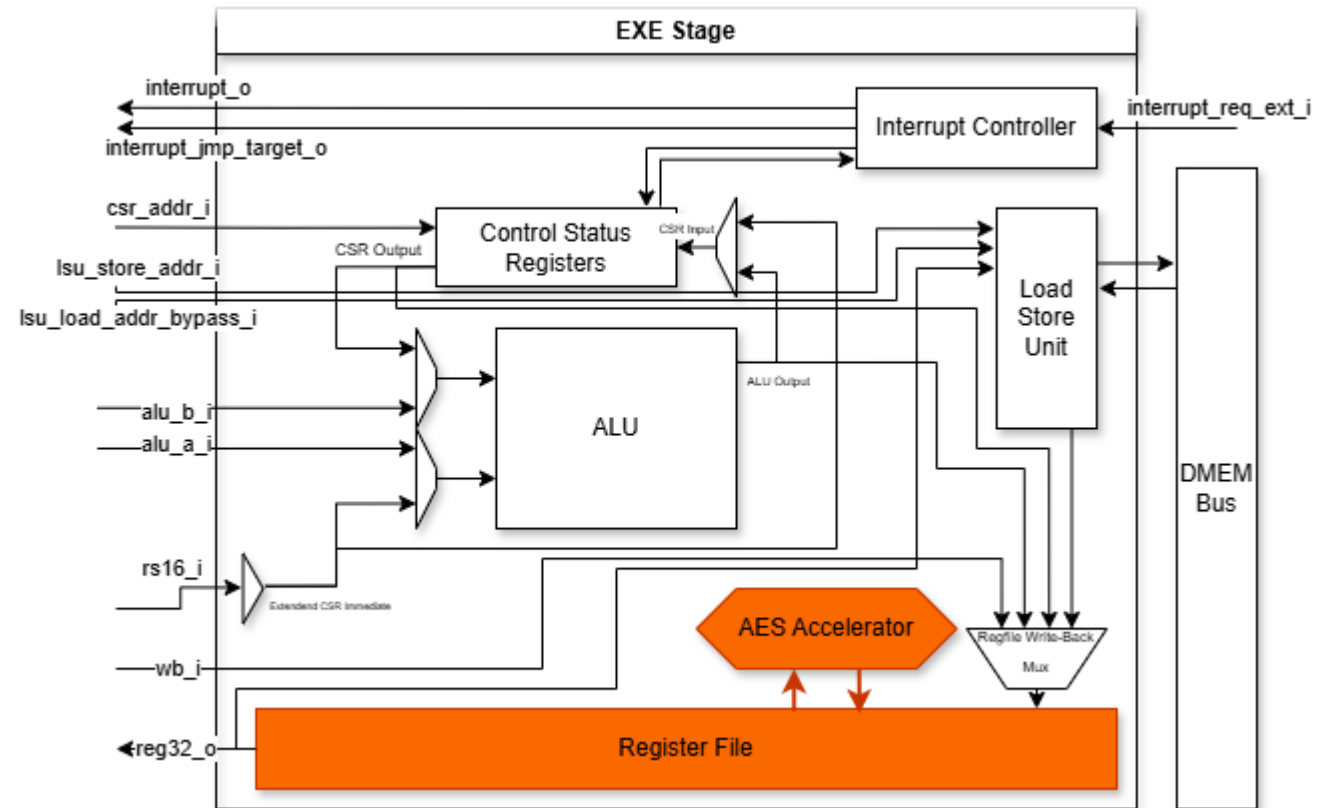
Integration & Software Interface

Parametric AES Accelerator

- Allows latency to be explicitly tuned for the target application.
- Area-Optimized Configuration (Minimal S-Boxes):
 - Latency: ~160 Cycles per block.
 - Low-power.
- High-Performance Configuration (16 S-Boxes):
 - Latency: ~11 Cycles per block.
 - High-throughput streaming.

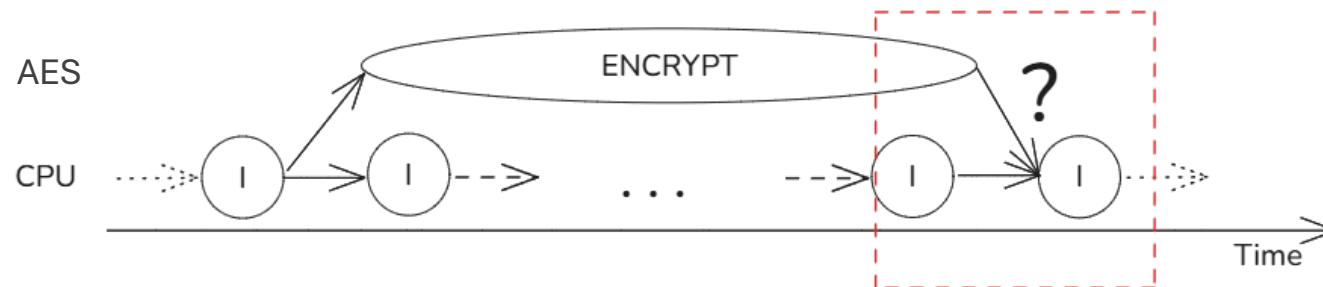
Integration

- Tightly coupled to the Core.
 - eliminates bus communication overhead.
 - area and power efficiency through hardware reuse.
- Using Registers x28-x31 as interface.
- Concurrent Execution.



Synchronization Methods

- Makes the core able to work with different kinds of accelerators or different accelerator modes/scenarios.
 - Polling.
 - For short variable time execution accelerators.
 - Waiting.
 - For short roughly constant time execution accelerators.
 - Accelerator interrupts.
 - For long and variable times execution accelerators.

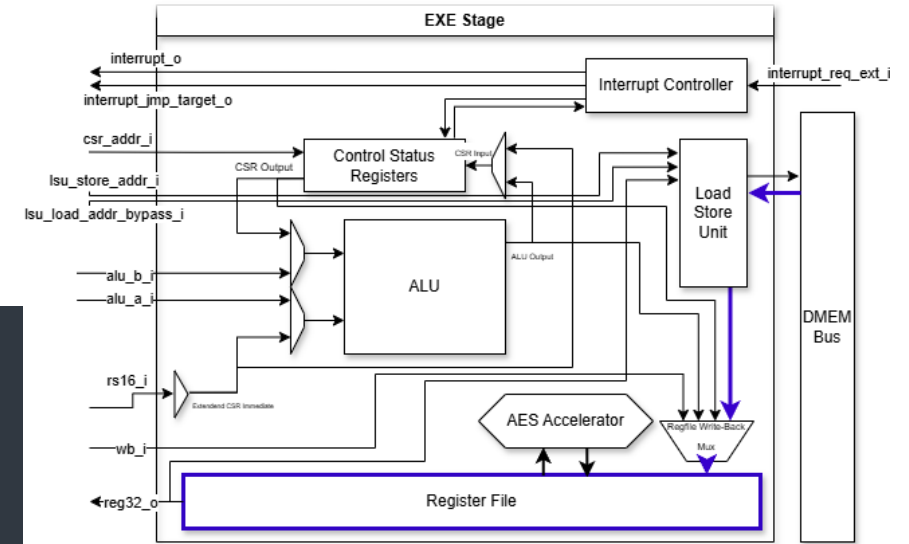


Software Interface

- Custom instructions for controlling the AES accelerator and for Waiting Mode.
- A CSR interface for Polling Mode.
- An interrupt interface for Interrupt Mode.

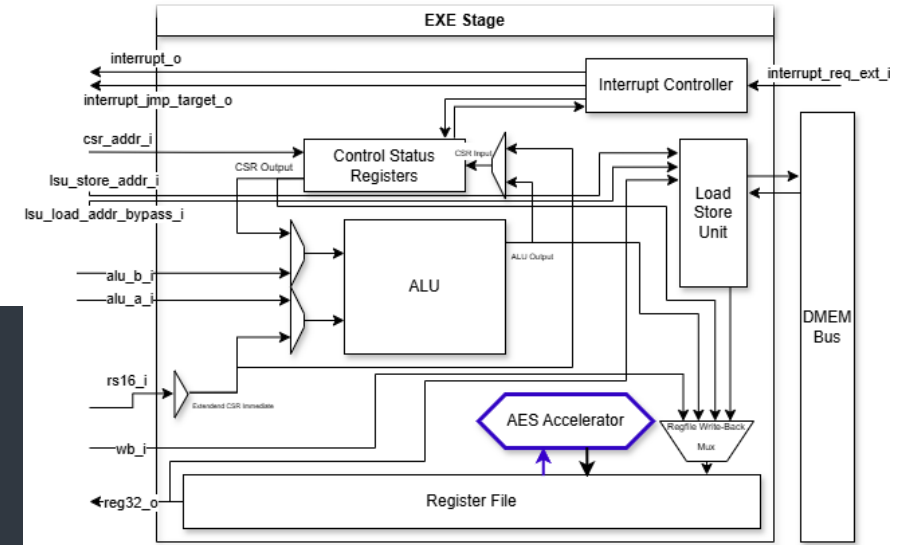
C Example

```
int main() {  
    // 1. Load key  
    LOAD_BLOCK_TO_REGS(key);  
    AES_LOAD_KEY();  
    // 2. Start first block  
    LOAD_BLOCK_TO_REGS(pts[0]);  
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.  
    // 3. Pipeline loop  
    for (int i = 1; i < N; i++) {  
        // Load NEXT plaintext while accelerator works on CURRENT block.  
        LOAD_BLOCK_TO_REGS(pts[i]);  
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)  
        AES_ENC_S();  
        // Store the results we just retrieved  
        STORE_BLOCK_FROM_REGS(cts[i-1]);  
    }  
    // 4. Drain the pipe (Retrieve last block)  
    AES_SYNC(); // Wait for the last block and retrieve it.  
    STORE_BLOCK_FROM_REGS(cts[N-1]);  
}
```



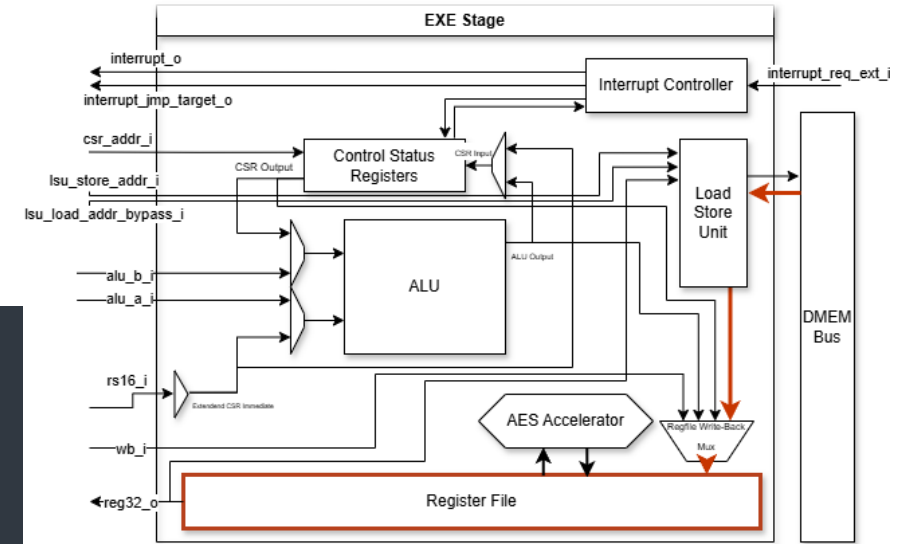
C Example

```
int main() {  
    // 1. Load key  
    LOAD_BLOCK_TO_REGS(key);  
    AES_LOAD_KEY();  
    // 2. Start first block  
    LOAD_BLOCK_TO_REGS(pts[0]);  
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.  
    // 3. Pipeline loop  
    for (int i = 1; i < N; i++) {  
        // Load NEXT plaintext while accelerator works on CURRENT block.  
        LOAD_BLOCK_TO_REGS(pts[i]);  
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)  
        AES_ENC_S();  
        // Store the results we just retrieved  
        STORE_BLOCK_FROM_REGS(cts[i-1]);  
    }  
    // 4. Drain the pipe (Retrieve last block)  
    AES_SYNC(); // Wait for the last block and retrieve it.  
    STORE_BLOCK_FROM_REGS(cts[N-1]);  
}
```

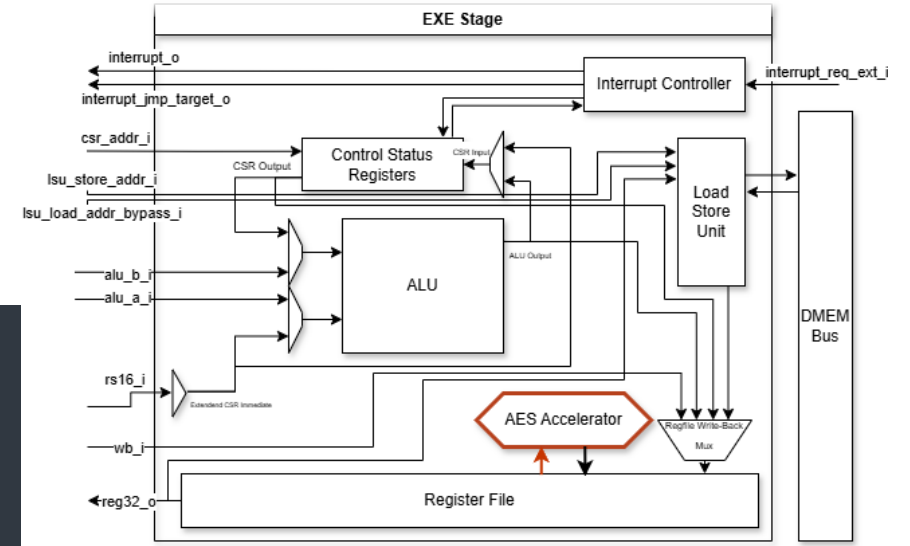


C Example

```
int main() {
    // 1. Load key
    LOAD_BLOCK_TO_REGS(key);
    AES_LOAD_KEY();
    // 2. Start first block
    LOAD_BLOCK_TO_REGS(pts[0]);
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.
    // 3. Pipeline loop
    for (int i = 1; i < N; i++) {
        // Load NEXT plaintext while accelerator works on CURRENT block.
        LOAD_BLOCK_TO_REGS(pts[i]);
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)
        AES_ENC_S();
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }
    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
}
```

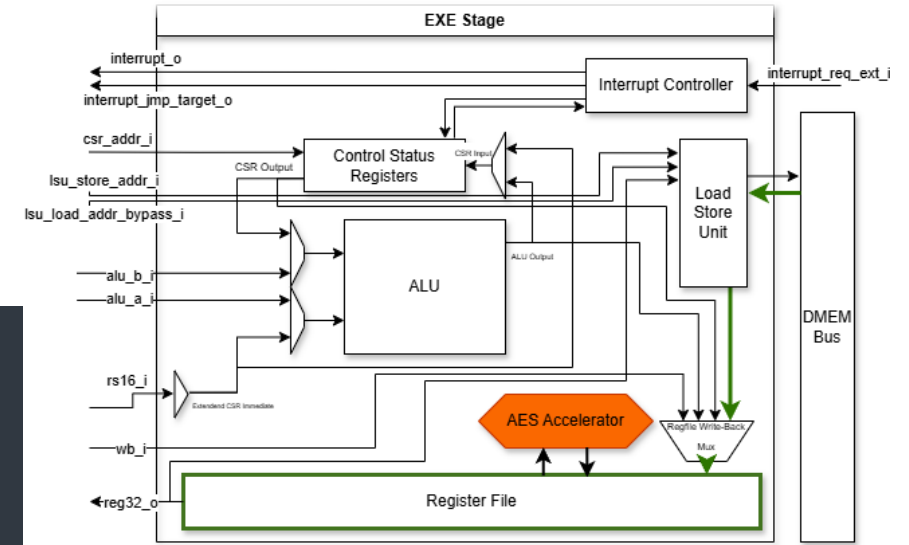



```
int main() {
    // 1. Load key
    LOAD_BLOCK_TO_REGS(key);
    AES_LOAD_KEY();
    // 2. Start first block
    LOAD_BLOCK_TO_REGS(pts[0]);
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.
    // 3. Pipeline loop
    for (int i = 1; i < N; i++) {
        // Load NEXT plaintext while accelerator works on CURRENT block.
        LOAD_BLOCK_TO_REGS(pts[i]);
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)
        AES_ENC_S();
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }
    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
}
```



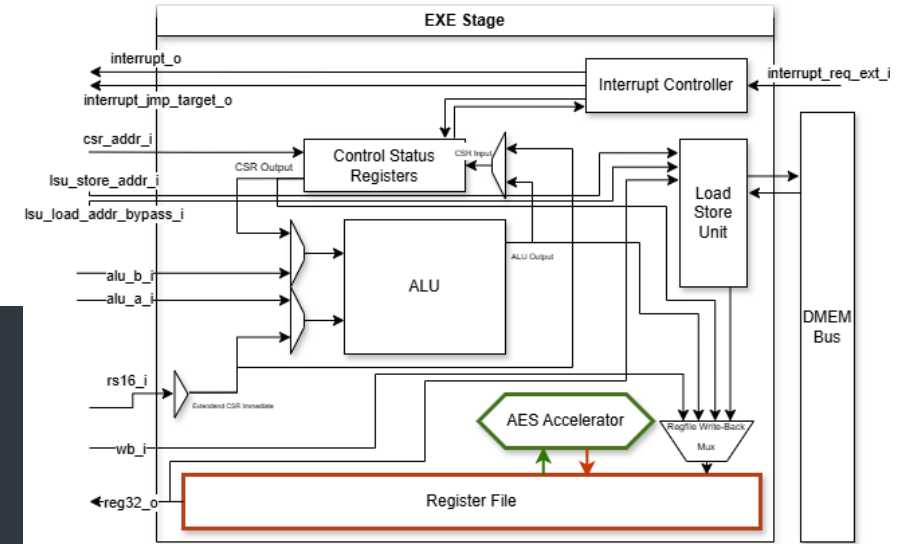
C Example

```
int main() {
    // 1. Load key
    LOAD_BLOCK_TO_REGS(key);
    AES_LOAD_KEY();
    // 2. Start first block
    LOAD_BLOCK_TO_REGS(pts[0]);
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.
    // 3. Pipeline loop
    for (int i = 1; i < N; i++) {
        // Load NEXT plaintext while accelerator works on CURRENT block.
        LOAD_BLOCK_TO_REGS(pts[i]);
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)
        AES_ENC_S();
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }
    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
}
```



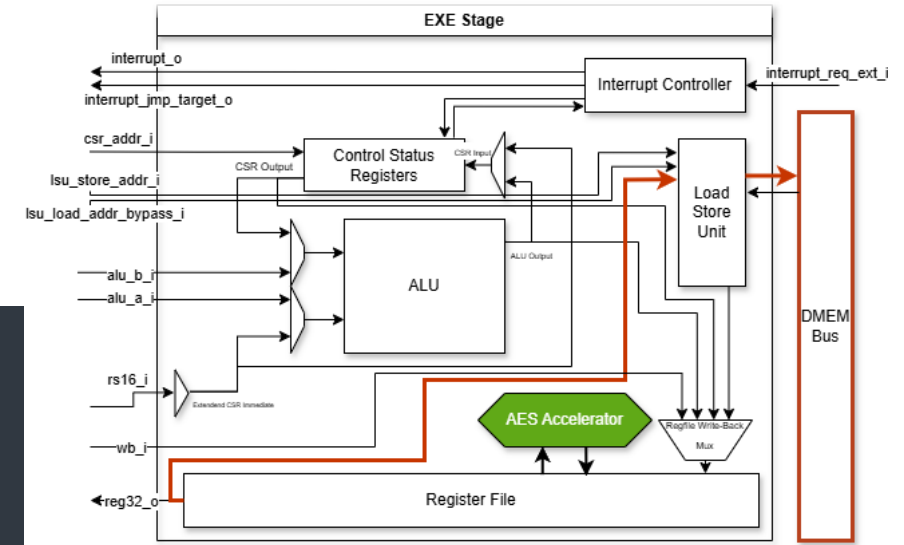
C Example

```
int main() {  
    // 1. Load key  
    LOAD_BLOCK_TO_REGS(key);  
    AES_LOAD_KEY();  
    // 2. Start first block  
    LOAD_BLOCK_TO_REGS(pts[0]);  
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.  
    // 3. Pipeline loop  
    for (int i = 1; i < N; i++) {  
        // Load NEXT plaintext while accelerator works on CURRENT block.  
        LOAD_BLOCK_TO_REGS(pts[i]);  
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)  
        AES_ENC_S();  
        // Store the results we just retrieved  
        STORE_BLOCK_FROM_REGS(cts[i-1]);  
    }  
    // 4. Drain the pipe (Retrieve last block)  
    AES_SYNC(); // Wait for the last block and retrieve it.  
    STORE_BLOCK_FROM_REGS(cts[N-1]);  
}
```



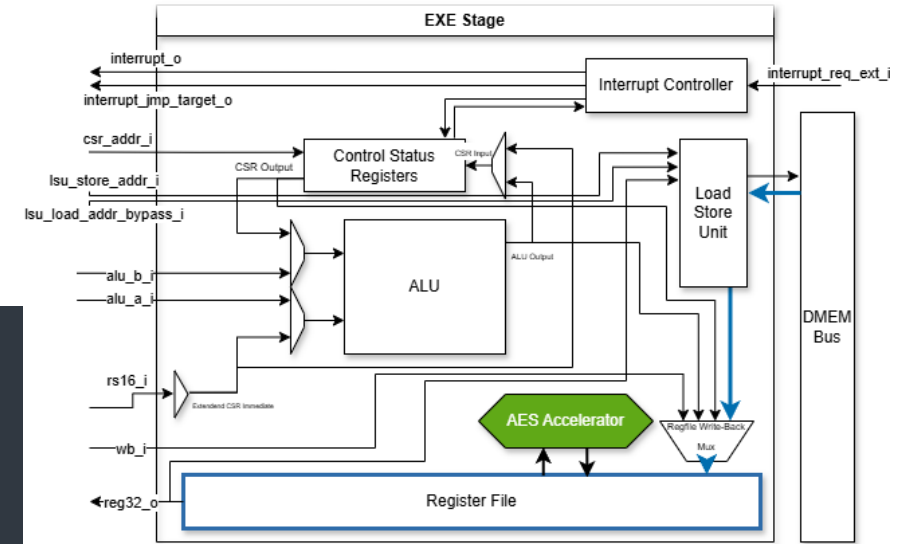
C Example

```
int main() {
    // 1. Load key
    LOAD_BLOCK_TO_REGS(key);
    AES_LOAD_KEY();
    // 2. Start first block
    LOAD_BLOCK_TO_REGS(pts[0]);
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.
    // 3. Pipeline loop
    for (int i = 1; i < N; i++) {
        // Load NEXT plaintext while accelerator works on CURRENT block.
        LOAD_BLOCK_TO_REGS(pts[i]);
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)
        AES_ENC_S();
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }
    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
}
```



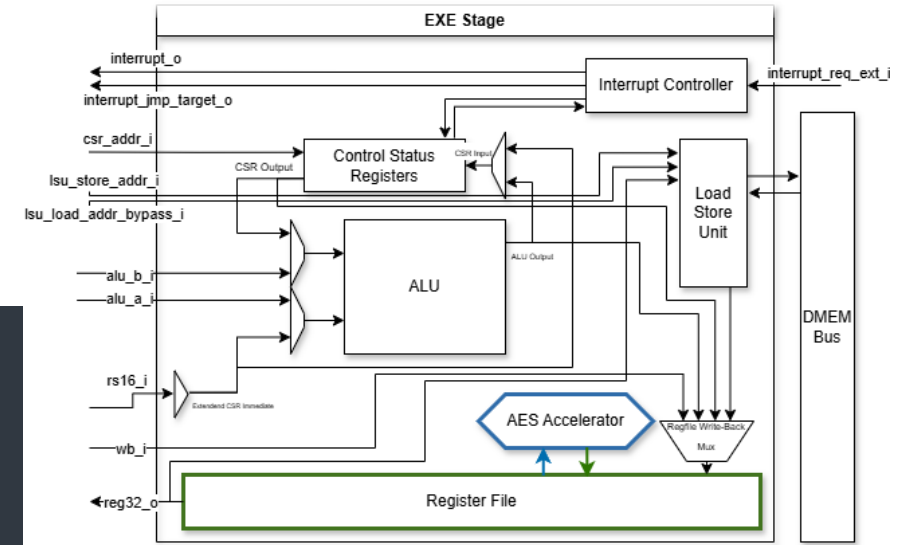
C Example

```
int main() {
    // 1. Load key
    LOAD_BLOCK_TO_REGS(key);
    AES_LOAD_KEY();
    // 2. Start first block
    LOAD_BLOCK_TO_REGS(pts[0]);
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.
    // 3. Pipeline loop
    for (int i = 1; i < N; i++) {
        // Load NEXT plaintext while accelerator works on CURRENT block.
        LOAD_BLOCK_TO_REGS(pts[i]);
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)
        AES_ENC_S();
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }
    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
}
```



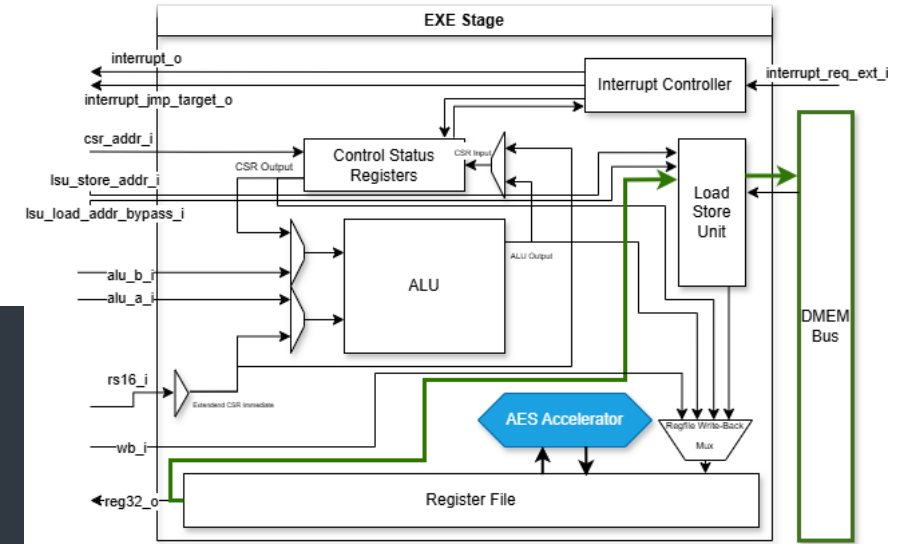
C Example

```
int main() {  
    // 1. Load key  
    LOAD_BLOCK_TO_REGS(key);  
    AES_LOAD_KEY();  
    // 2. Start first block  
    LOAD_BLOCK_TO_REGS(pts[0]);  
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.  
    // 3. Pipeline loop  
    for (int i = 1; i < N; i++) {  
        // Load NEXT plaintext while accelerator works on CURRENT block.  
        LOAD_BLOCK_TO_REGS(pts[i]);  
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)  
        AES_ENC_S();  
        // Store the results we just retrieved  
        STORE_BLOCK_FROM_REGS(cts[i-1]);  
    }  
    // 4. Drain the pipe (Retrieve last block)  
    AES_SYNC(); // Wait for the last block and retrieve it.  
    STORE_BLOCK_FROM_REGS(cts[N-1]);  
}
```



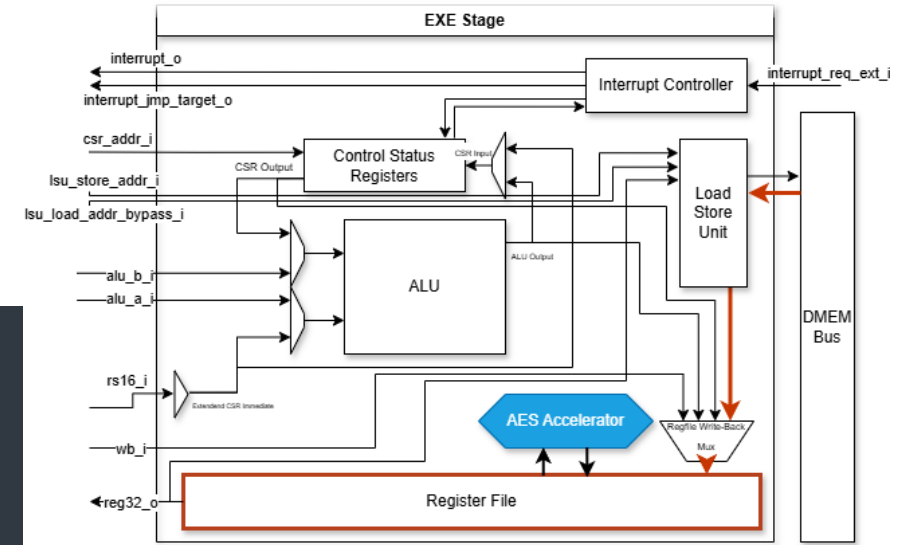
C Example

```
int main() {  
    // 1. Load key  
    LOAD_BLOCK_TO_REGS(key);  
    AES_LOAD_KEY();  
    // 2. Start first block  
    LOAD_BLOCK_TO_REGS(pts[0]);  
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.  
    // 3. Pipeline loop  
    for (int i = 1; i < N; i++) {  
        // Load NEXT plaintext while accelerator works on CURRENT block.  
        LOAD_BLOCK_TO_REGS(pts[i]);  
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)  
        AES_ENC_S();  
        // Store the results we just retrieved  
        STORE_BLOCK_FROM_REGS(cts[i-1]);  
    }  
    // 4. Drain the pipe (Retrieve last block)  
    AES_SYNC(); // Wait for the last block and retrieve it.  
    STORE_BLOCK_FROM_REGS(cts[N-1]);  
}
```



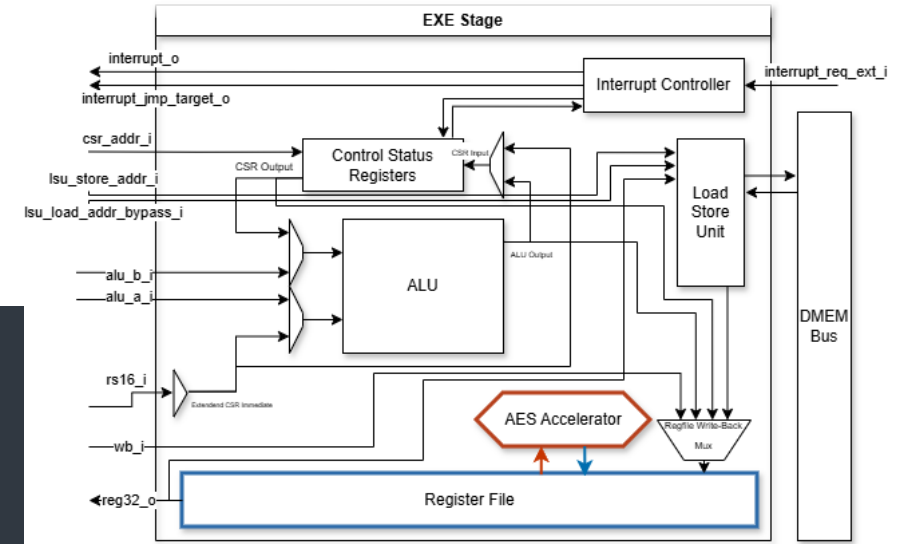
C Example

```
int main() {  
    // 1. Load key  
    LOAD_BLOCK_TO_REGS(key);  
    AES_LOAD_KEY();  
    // 2. Start first block  
    LOAD_BLOCK_TO_REGS(pts[0]);  
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.  
    // 3. Pipeline loop  
    for (int i = 1; i < N; i++) {  
        // Load NEXT plaintext while accelerator works on CURRENT block.  
        LOAD_BLOCK_TO_REGS(pts[i]);  
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)  
        AES_ENC_S();  
        // Store the results we just retrieved  
        STORE_BLOCK_FROM_REGS(cts[i-1]);  
    }  
    // 4. Drain the pipe (Retrieve last block)  
    AES_SYNC(); // Wait for the last block and retrieve it.  
    STORE_BLOCK_FROM_REGS(cts[N-1]);  
}
```



C Example

```
int main() {
    // 1. Load key
    LOAD_BLOCK_TO_REGS(key);
    AES_LOAD_KEY();
    // 2. Start first block
    LOAD_BLOCK_TO_REGS(pts[0]);
    AES_ENC(); // Copies x28-x31 to accel and starts. Regs are now free.
    // 3. Pipeline loop
    for (int i = 1; i < N; i++) {
        // Load NEXT plaintext while accelerator works on CURRENT block.
        LOAD_BLOCK_TO_REGS(pts[i]);
        // Swap: Wait for accel to finish, grab Ciphertext(i - 1), give Plaintext(i)
        AES_ENC_S();
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }
    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
}
```



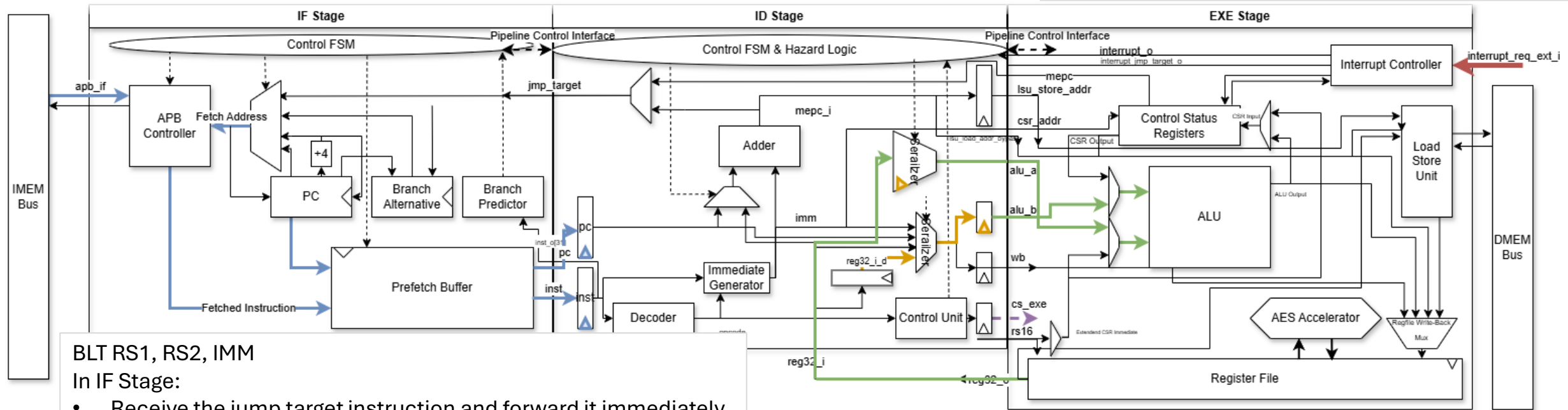
Core Verification

- Module-Level Fuzzing for critical modules in SystemVerilog.
- Directed Pipeline Verification in SystemVerilog.
- System-Level Software Verification using compiled C Programs.
 - With random IMEM/DMEM delays and occasional external machine interrupts.

- Questions?

Datapath Walkthrough - Interrupt

External Interrupt Request received.
Will be acted on when ID stage is not in the middle of executing an instruction.



BLT RS1, RS2, IMM

In IF Stage:

- Receive the jump target instruction and forward it immediately to ID stage.

In ID Stage:

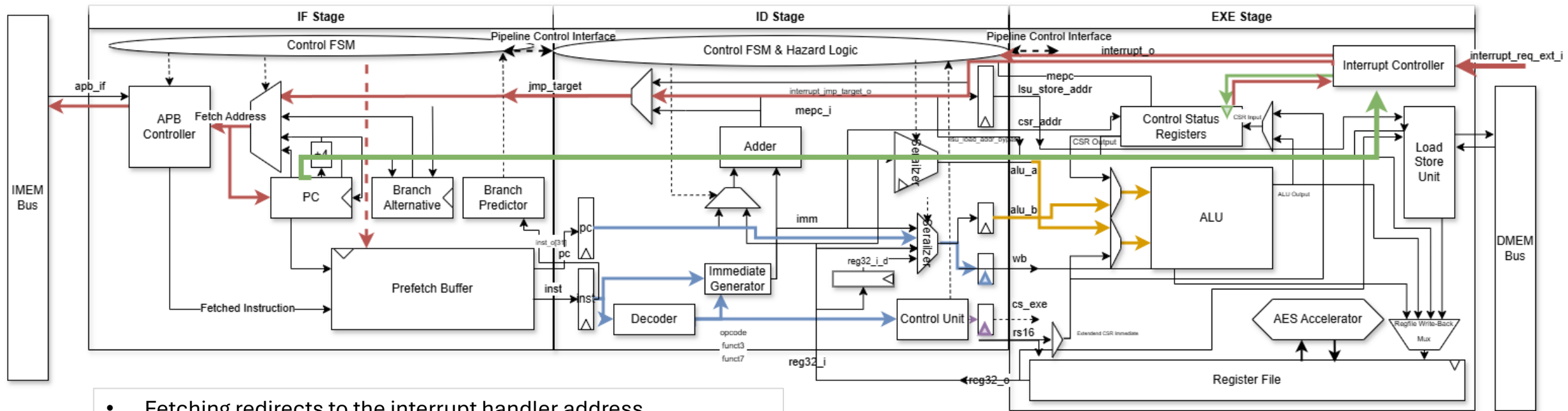
- Continue half forwarding procedure.

BLT RS1, RS2, IMM

In EXE Stage:

- Compare first halves. Signal IF stage incase of early resolution.

Datapath Walkthrough - Interrupt



- Fetching redirects to the interrupt handler address.
- The return from interrupt address is stored in a CSR.
- The instruction currently executed in ID stage continues to execute if possible.

BLT RS1, RS2, IMM

In EXE Stage:

- Compare second halves. Signal IF stage the resolution.